	Metadatos
Autor	Francisco J. Gallego-Durán
Actualizado	20/septiembre/2024
Licencia	CC-4.0-NC-ND
Lectura Estimada	35 minutos

Gameboy: Cómo crear gráficos a mano. Ejemplo.

En la Gameboy, todos los gráficos se basan en un sistema de tiles (valdosas) de 8x8 píxeles. Son utilizadas tanto para dibujar los tilemaps de fondo y las ventanas de primer plano, como para las fuentes y también los sprites de personajes, enemigos y objetos.

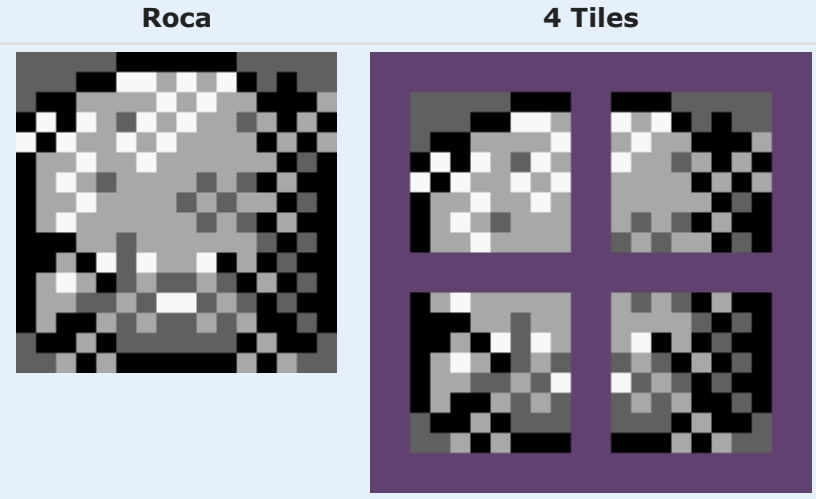
Los tiles en Gameboy tienen un formato concreto y sus datos deben escribirse en la Zona de Tiles de la VRAM. A continuación creamos cuatro tiles paso a paso, para que se entienda todo el proceso.

Utilizaremos como ejemplo una roca de los juegos Pokemon Azul y Rojo. Estos tiles son propiedad de Nintendo y que aquí se usan sólo como ejemplo con fines educativos.

Roca de Pokemon Rojo/Azul

La Figura 1 muestra una roca que se puede encontrar en mazmorras en Pokemon Rojo y Azul, descompuesta en los cuatro tiles de 8x8 píxeles que la forman.

Figura 1



Roca de Pokemon Rojo y Azul y sus 4 tiles

Sabiendo que tenemos 8x8 píxeles y que cada píxel puede tener hasta 4 colores, necesitamos 2 bits por cada píxel, y eso nos da un total de 16 bytes para un único tile. La Tabla 1 resume este cálculo paso a paso.

Concepto	Espacio requerido	Cálculo
4 colores por píxel	2 bits	$2^2 = 4$ (2 bits)

Concepto	Espacio requerido	Cálculo
1 fila (8 píxeles de ancho)	16 bits (2 bytes)	`2 x 8  = 16 bits (2 bytes)`
8 filas (8x8 píxeles)	128 bits (16 bytes)	`16 x 8 = 128 bits (16 bytes)`

Pero necesitamos saber que significado tienen esos bits (***formato***) para que la Gameboy los entienda y convierta en los cuatro tiles de la Figura 1. Los detalles de este ***formato*** los tienes explicados en estos dos artículos: (1: Pandocs VRAM Tile Data), (2: GB - VRAM Tile Data). Ahora, vamos a mostrarlo de forma práctica, con cada uno de los cuatro tiles.

Teniendo en cuenta la paleta

Antes de convertir los tiles, necesitamos tener ajustada la Paleta de Fondo de la Gameboy. La paleta es la información que usa la Gameboy para decidir a qué color corresponde cada valor de un pixel. Es un array de cuatro posiciones, identificando los cuatro colores posibles. La Tabla 2 muestra cuatro paletas distintas de ejemplo. Cada índice de la paleta puede representar un color u otro. Por ejemplo, el valor 3 (`11`) representará el color Negro si activamos la Paleta A, pero representará el Gris Oscuro si activamos la Paleta B. Igualmente, el valor 1 (`01`) indicará color Blanco si activamos la Paleta B, o Gris claro si activamos la Paleta D.

Índice	Paleta A	Paleta B	Paleta C	Paleta D
3 (`11`)	Negro	Gris Oscuro	Negro	Negro
2 (`10`)	Gris Oscuro	Gris Claro	Negro	Gris Oscuro
1 (`01`)	Gris Oscuro	Blanco	Negro	Gris Claro
0 (`00`)	Negro	Blanco	Blanco	Blanco

En la Gameboy tenemos una única Paleta para el Fondo y dos paletas para los objetos/sprites.

¿Para qué sirve la paleta?

Aunque parezca trabajo extra, la paleta es muy útil para hacer multitud de efectos gráficos con muy poco coste computacional. Por ejemplo, imagina que queremos hacer un clásico Efecto de Fade-out o Fade-in (entrada y salida suavizada de los colores). Si queremos que los colores desaparezcan (Fade-out) lentamente, en un paso el negro tendría que convertirse en gris oscuro, en el siguiente paso en gris claro, y luego en blanco. Eso nos llevaría a tener que cambiar todos los píxeles de todos los tiles uno a uno.

Disponiendo de una paleta, hacer que los colores desaparezcan poco a poco ((Fade-out)) sólo necesita cambiar la paleta, sin tocar ninguno de los píxeles. Si estamos usando la Paleta A de la Tabla 2, todos los píxeles de color 3 (`11`) son negros. Con sólo cambiar a la Paleta B, todos esos píxeles se vuelven automáticamente Gris Oscuro. Lo que cambiamos es la definición de qué color es el número 3 (`11`): antes era negro, ahora es gris oscuro. De la misma forma, sólo habría que ir cambiando la definición del color 3 (`11`) a gris claro y luego a blanco. El efecto se completaría y no haría falta tocar ningún tile ni sprite.

Con esta misma idea podemos hacer efectos de flash o efectos que simulen movimiento de agua o fluídos, simplemente cambiando las definiciones de la paleta. Por eso es tan útil disponer de una paleta.

Definiendo el primer tile

Sabiendo cómo funciona la paleta, hacemos un primer paso sencillo de convertir los píxeles en números. En este paso, para cada píxel miramos su color en la paleta, y anotamos el índice. En la Tabla 3 partimos de la imagen del tile, que la convertimos en una matriz de letras que identifican el color de cada píxel. Usando la paleta, que asocia a cada color un índice, convertimos las letras en números, en binario. Los píxeles se muestran agrupados de cuatro en cuatro por claridad.

Tile	Píxeles	Paleta	Pixeles en binario
	GGGG GNNN GGGN NBBq GNNq qqqB NBNB qGBq BNBq qBqB NqqB qqBq NqBq Gqqq NggB gggg	11 : **N**egro 10 : **G**ris Oscuro 01 : **g**ris Claro 00 : **B**lanco	10 10 10 10 · 10 11 11 11 10 10 10 11 · 11 00 00 01 10 11 11 01 · 01 01 01 00 11 00 11 00 · 01 10 00 01 00 11 00 01 · 01 00 01 00 11 01 01 00 · 01 01 00 01 11 01 00 01 · 10 01 01 01 11 01 01 00 · 01 01 01 01
Imagen del tile	Colores por píxel **N**egro, **G**ris_Oscuro, **g**ris_Claro, **B**lanco	Paleta de cuatro colores usada. *Índice : Color*	Colores por píxel en binario. Cada píxel usa el índice de paleta de su color

Lamentablemente, aunque esta codificación tan clara y directa es la que utiliza la Gameboy, no almacena los datos en su VRAM de Tiles de esta forma. Los almacena intercalados, en un formato llamado ***bitplanes*** (planos de bits). Para entenderlo bien, la Tabla 4 muestra cómo los píxeles en binario se separan en dos ***bitplanes*** (alto y bajo) y se ordenan en memoria como | bp.bajo | bp.alto | en la codificación final.

	binario	hexadecimal
Píxeles	10 10 10 10 10 11 11 11	
bitplane **alto**	1 1 1 1 1 1 1 1	$FF
bitplane **bajo**	0 0 0 0 0 1 1 1	$07
Codificación GB	00000111 11111111	$07 $FF

Un ***bitplane*** son los bits que ocupan la misma posición en varios números consecutivos. Puesto que tenemos ocho números de dos bits (un número por píxel), tenemos dos ***bitplanes***: uno con todos los bits altos de cada número, y otro con todos los bits bajos. Así, el ***bitplane alto*** en la Tabla 4 son todos los bits altos de cada número (píxel) juntos. Lógicamente, el ***bitplane bajo*** son todos los bits bajos juntos.

La Tabla 5 muestra cómo se convierten todos los píxeles del tile en sus bitplanes (***bp***) bajo y alto. La última columna los muestra en hexadecimal, tal como se verán en la memoria en el desensamblado. Así, tal como quedan en la Tabla 5, es como los espera la Gameboy en la VRAM de Tiles.

Tile	Pixeles en binario	bp.BAJO	bp.ALTO	Codificación para GB - VRAM
	10 10 10 10 · 10 11 11 11 10 10 10 11 · 11 00 00 01 10 11 11 01 · 01 01 01 00 11 00 11 00 · 01 10 00 01 00 11 00 01 · 01 00 01 00 11 01 01 00 · 01 01 00 01 11 01 00 01 · 10 01 01 01 11 01 01 00 · 01 01 01 01	0000·0111 0001·1001 0111·1110 1010·1001 0101·1010 1110·1101 1101·0111 1110·1111	1111·1111 1111·1000 1110·0000 1010·0100 0100·0000 1000·0000 1000·1000 1000·0000	$03 $FF $19 $F8 $7E $E0 $A9 $A4 $5A $40 $ED $80 $D7 $88 $EF $80

Con esta codificación ya podemos decirle a la Gameboy que el Tile nº1 (dirección de memoria `$8010` en la VRAM de Tiles) será, a partir de ahora, esta parte superior izquierda de una roca. Sólo tenemos que copiar los dieciséis bytes seguidos, tal como quedan en la Tabla 6. Así, por orden, cada pareja de bytes son los 2 byte planes (bajo-alto) que definen una fila, empezando por la fila 1 del tile, empezando de arriba hacia abajo.

Dirección	00 01	02 03	04 05	06 07	08 09	0A 0B	0C 0D	0E 0F
$8010	03 FF	19 F8	7E E0	A9 A4	5A 40	ED 80	D7 88	EF 80
	Fila 1	*Fila 2*	*Fila 3*	*Fila 4*	*Fila 5*	*Fila 6*	*Fila 7*	*Fila 8*

Definir los otros tres tiles

Te recomiendo que practiques con los otros tres tiles y calcules su codificación en ***bitplanes*** por tu cuenta. Cuando lo hayas hecho, vuelve aquí y comprueba si te han salido los mismos valores. Repasa y prueba un par de veces hasta que sepas cómo hacerlo. Una vez termines, ya sabes cómo crear gráficos para Gameboy. A partir de aquí, puedes utilizar cualquier programa que los calcule por ti, para ahorrarte el tiempo. También, como sabes cómo funciona, puedes arreglar problemas o crearte tus propias herramientas.

Codificación de los otros tres tiles

Suponiendo que usamos la misma paleta para los otros tres tiles y que sus números de tile son el 2, 3 y 4, quedarían en la VRAM de Tiles como muestra la Tabla 7.

Dirección	00 01	02 03	04 05	06 07	08 09	0A 0B	0C 0D	0E 0F
$8020	E0 FF	54 1F	BF 0E	6F 15	FF 0A	FD 07	AF 5B	5D A7
$8030	BF 80	FB E4	F1 D4	DA 8D	E4 9A	FA B5	78 EF	3F D7
$8040	AF 5B	F5 0F	BB 2F	5D B7	2B 5F	5D AF	1E F7	FC EB
	Fila 1	*Fila 2*	*Fila 3*	*Fila 4*	*Fila 5*	*Fila 6*	*Fila 7*	*Fila 8*

Dibujando la roca en la Gameboy

Ahora que ya tenemos la codificación, sólo nos queda dibujar la roca en la Gameboy. Para eso, tendremos que crear un programa que incluya todos los bytes que hemos calculado y los copie a la Zona de tiles de la VRAM. Una vez estén allí, podremos modificar la Zona del Tilemap de la VRAM para poner los números de cada tile en la esquina superior derecha. Si lo hacemos bien, debe quedarnos como en la Figura 2.

Figura 2



Roca hecha de cuatro tiles, en la Gameboy

Pasos a seguir

Sigue exactamente estos pasos para incluir la roca en tu programa y luego dibujarla.

1. Añade los datos a tu ROM.
2. Haz una función que copie los datos de la ROM a la Zona de tiles de la VRAM. Tal como hemos hecho, de `$8010` a `$804F` (Los números de tile 1, 2, 3 y 4).
3. Haz un pequeño código para poner los números de tile 1, 2, 3 y 4 en las posiciones (0,0), (1,0), (1,0) y (1,1) del tilemap.

> ⚠ **¡OJO!**
>
> Sólo puedes copiar datos a la VRAM durante VBLANK o HBLANK. Si no te da tiempo, puedes usar varios VBLANK/HBLANK. También puedes apagar la pantalla, copiar y luego volver a encenderla.
>
> Además, recuerda que el apagado de pantalla ***DEBE*** hacerse al comienzo de VBLANK. En caso contrario, se puede ***DAÑAR*** la pantalla de la Gameboy.

Cómo se añaden los datos a la ROM

Puedes añadir datos a la ROM usando el lenguaje ensamblador de RGBDS. Para empezar, todo lo que se produce después de ensamblar y enlazar tu programa es una ROM con datos binarios. Por tanto, todo lo que haces en tu programa es, en sí mismo, añadir datos a la ROM.

Cuando escribes un código en lenguaje ensamblador, el programa ensamblador lo traduce a números: códigos de operación (opcodes) y parámetros (datos). Por ejemplo, el Listado 1 muestra cómo tres instrucciones en lenguaje ensamblador son convertidas a seis bytes en código máquina, que terminan escritos en la ROM.

Traducción de ensamblador a código máquina

```
1    SECTION "Código", ROM0[$200]
2    ;; Lenguaje
3    ;; Ensamblador  ;; Código Máquina
4    ld  a, $19      ;; 3E 19
5    ld hl, $9800    ;; 21 00 98
6    ld [hl], a      ;; 77
```

Se suele usar la palabra ***emitir*** para indicar cómo el programa ensamblador ***añade*** bytes al binario resultante (nuestra ROM, en este caso). Puesto que la directiva **SECTION** le indica que todo lo que viene a continuación debe estar en la posición de memoria `$200` de la ***ROM0***, los seis bytes emitidos al traducir las tres instrucciones se añadirán a las posiciones `$200-$205` del binario resultante (la ROM).

Pues bien, el programa ensamblador, además de traducir instrucciones de lenguaje ensamblador a código máquina, nos permite ***emitir*** bytes directamente al binario resultante. Para eso, utilizamos la directiva **DB** (***Define Byte***). De hecho, el programa anterior podemos crearlo "a mano", en código máquina, en vez de en ensamblador, usando **DB** como en el Listado 2.

Programando directamente en código máquina

```
1    SECTION "Código", ROM0[$200]
2    ;; Código
3    ;; Máquina        ;; Lenguaje Ensamblador
4    DB $3E, $19       ;; ld  a, $19
5    DB $21, $00, $98  ;; ld hl, $9800
6    DB $77            ;; ld [hl], a
```

De ambas formas, terminaremos ***exactamente*** con la misma ROM, ya que serán emitidos los mismos bytes al binario resultante.

Esto nos permite introducir todo tipo de código y datos cuando queramos. Por ejemplo, para introducir los datos de nuestro primer tile podemos hacerlo como en el Listado 3.

Añadiendo los datos de un tile a nuestra ROM

```
1    SECTION "Código", ROM0[$200]
2    ;; Lenguaje
3    ;; Ensamblador  ;; Código Máquina
4    ld  a, $19      ;; 3E 19
5    ld hl, $9800    ;; 21 00 98
6    ld [hl], a      ;; 77
7    di              ;; F3
8    halt            ;; 76
9
10   ;; 16 bytes del primer tile de la roca.
11   ;; Ponemos una etiqueta delante para luego poder saber en qué
12   ;; dirección de memoria terminan estos bytes. La etiqueta será el
13   ;; valor de la dirección de memoria del byte que vale $03 (el primero)
14   ;; Así podremos hacer cosas como
15   ;;  ld hl, tile_roca      ;; HL = direccion de memoria del primer byte del tile ($03)
16   ;;  ld a, [tile_roca]     ;; A = $03 (leemos el primer byte del tile de la memoria)
17   ;;  ld a, [tile_roca + 2] ;; A = $19 (leemos el tercer byte del tile de la memoria)
18   ;;....
19   tile_roca:
20   DB $03, $FF, $19, $F8, $7E, $E0, $A9, $A4, $5A, $40, $ED, $80, $D7, $88, $EF, $80
```

Ahora que tenemos los datos del tile en nuestra ROM, podemos copiarlos cuando queramos a la Zona de tiles de la VRAM para que la Gameboy lo lea y nos permita dibujarlo en pantalla.

References

1. Gameboy Pandocs VRAM Tile Data